



(Theory) Workflow Orchestration



Credits to the notebook by @ManuelGuerra1987:

https://github.com/ManuelGuerra1987/data-engineering-zoomcamp-notes/tree/main/2_Workflow-Orchestration-AirFlow

Workflow Orchestration using Airflow DE Zoomcamp 2022:

https://github.com/DataTalksClub/data-engineering-zoomcamp/tree/main/cohorts/2022/week_2_data_ingestion

▼ Recap Module 1

In the previous module we created a script that:

- Downloads a file and unzips it.
- Saves it locally as a CSV file.
- Loads and inserts the data into a PostgreSQL database.

▼ Source Code:

```
import os
import argparse

from time import time

import pandas as pd
from sqlalchemy import create_engine

def main(params):
    user = params.user
    password = params.password
    host = params.host
    port = params.port
```

```

db = params.db
table_name = params.table_name
url = params.url

# the backup files are gzipped, and it's important to keep the correct extension
# for pandas to be able to open the file
if url.endswith('.csv.gz'):
    csv_name = 'output.csv.gz'
else:
    csv_name = 'output.csv'

os.system(f"wget {url} -O {csv_name}")

engine = create_engine(f'postgresql://{user}:{password}@{host}:{port}/{db}')

df_iter = pd.read_csv(csv_name, iterator=True, chunksize=100000)

df = next(df_iter)

df.tpep_pickup_datetime = pd.to_datetime(df.tpep_pickup_datetime)
df.tpep_dropoff_datetime = pd.to_datetime(df.tpep_dropoff_datetime)

df.head(n=0).to_sql(name=table_name, con=engine, if_exists='replace')

df.to_sql(name=table_name, con=engine, if_exists='append')

while True:

    try:
        t_start = time()

        df = next(df_iter)

        df.tpep_pickup_datetime = pd.to_datetime(df.tpep_pickup_datetime)
        df.tpep_dropoff_datetime = pd.to_datetime(df.tpep_dropoff_datetime)

        df.to_sql(name=table_name, con=engine, if_exists='append')

        t_end = time()

        print('inserted another chunk, took %.3f second' % (t_end - t_start))

    except StopIteration:
        print("Finished ingesting data into the postgres database")
        break

if __name__ == '__main__':
    parser = argparse.ArgumentParser(description='Ingest CSV data to Postgres')

    parser.add_argument('--user', required=True, help='user name for postgres')
    parser.add_argument('--password', required=True, help='password for postgres')
    parser.add_argument('--host', required=True, help='host for postgres')
    parser.add_argument('--port', required=True, help='port for postgres')
    parser.add_argument('--db', required=True, help='database name for postgres')
    parser.add_argument('--table_name', required=True, help='name of the table where we will write the results to')
    parser.add_argument('--url', required=True, help='url of the csv file')

```

```
args = parser.parse_args()
```

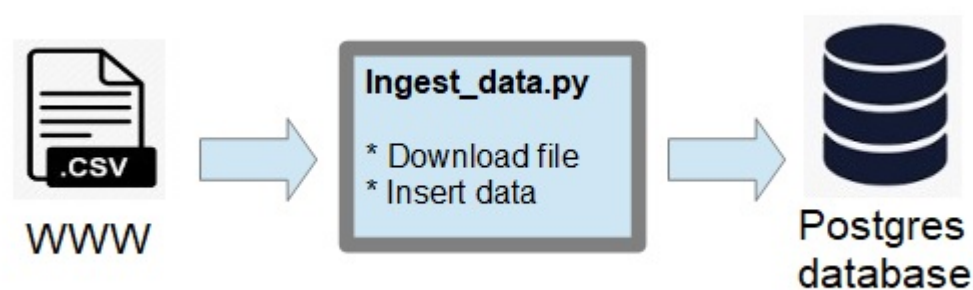
```
main(args)
```

```
URL="https://github.com/DataTalksClub/nyc-tlc-data/releases/download/yellow/yellow_tripdata_2021-01.csv.gz"
```

```
docker build -t taxi_ingest:v001 .
```

```
docker run -it \  
  --network=pg-network \  
  taxi_ingest:v001 \  
  --user=root \  
  --password=root \  
  --host=pg-database \  
  --port=5432 \  
  --db=ny_taxi \  
  --table_name=yellow_taxi_trips \  
  --url=${URL}
```

This is what our "pipeline" looked like:



▼ Masalah dengan Script Saat Ini

Kode yang kita lihat adalah contoh **cara yang kurang efisien** untuk membuat pipeline data. Script ini melakukan dua hal sekaligus:

1. Mengunduh file CSV dari internet
2. Memasukkan data ke database PostgreSQL

Beberapa masalah dengan pendekatan ini:

- Jika unduhan berhasil tetapi proses insert ke database gagal, kita harus mengulangi seluruh proses termasuk mengunduh data lagi
- Setiap kali menguji script, file CSV akan diunduh ulang (pemborosan waktu dan bandwidth)
- Sulit untuk menambahkan mekanisme retry dan penanganan kesalahan

▼ Struktur Pipeline yang Lebih Baik

Solusinya adalah **memisahkan proses** menjadi dua tugas terpisah:

1. **Tugas Pertama:** Mengunduh dan menyimpan file CSV
2. **Tugas Kedua:** Membaca file CSV dan memasukkan data ke PostgreSQL

Dengan struktur ini, jika tugas kedua gagal, kita tidak perlu mengunduh file lagi. Tugas kedua hanya dijalankan jika tugas pertama berhasil.

▼ Pentingnya Orkestrator Workflow

Inilah mengapa kita membutuhkan **orkestrator workflow** yang dapat:

- Mengatur urutan eksekusi tugas berdasarkan dependensi
- Menyediakan mekanisme retry jika tugas gagal

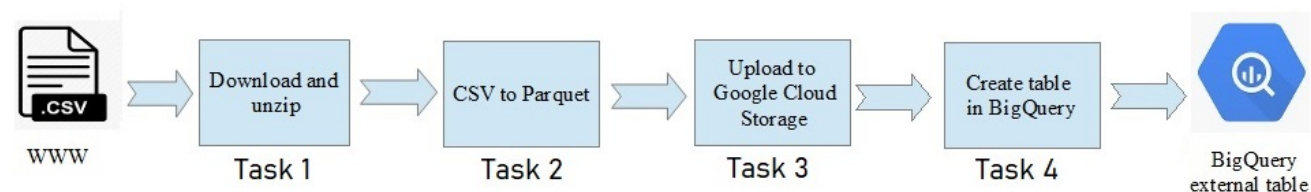
- Mencatat log dan riwayat eksekusi
- Menjadwalkan dan memantau alur kerja

▼ Pipeline yang Lebih Kompleks

Ke depan, kita akan bekerja dengan pipeline yang lebih kompleks:

1. Mengekstrak data dari web
2. Mengkonversi CSV ke format parquet (lebih efisien)
3. Mengunggah file ke Google Cloud Storage (data lake)
4. Membuat tabel eksternal di Google BigQuery (data warehouse)

Pipeline ini menunjukkan alur kerja data modern dengan pemisahan tugas yang jelas dan dependensi antar langkahnya.



▼ Perbedaan CSV vs Parquet

◆ CSV (Comma-Separated Values)

CSV adalah format **plain text** yang menyimpan data dalam bentuk baris dan kolom, di mana kolom dipisahkan oleh koma (atau delimiter lain seperti `;`, `\t`).

Contoh CSV:

```

name,age,city
Alice,30,New York
Bob,25,London
  
```

- Mudah dibaca oleh manusia
- Didukung oleh hampir semua tools (Excel, Python, SQL, dll.)
- Tipe data disimpan sebagai **string**
- Tidak efisien dalam penyimpanan atau akses data besar

◆ Parquet

Parquet adalah format **biner kolumnar** yang dikembangkan oleh Apache. Biasanya digunakan di ekosistem Hadoop, Spark, dan tools seperti AWS Athena, BigQuery, dll.

Ciri-ciri Parquet:

- **Kolumnar:** Menyimpan data per kolom, bukan per baris seperti CSV
- Mendukung **kompresi dan encoding** per kolom
- Memiliki **schema** internal (data type-aware)
- Cepat untuk query dan analisis data besar

```

Column: name   → [Alice, Bob, Charlie]
Column: age    → [30, 25, 35]
Column: city   → [New York, London, Sydney]
  
```

Dan bentuk fisiknya **tidak bisa dibaca langsung** karena disimpan sebagai file binary + metadata schema.

File `.parquet` ini cuma bisa dilihat pakai tools atau program khusus seperti:

- Python (pandas, pyarrow, fastparquet)

- Apache Spark
- AWS Athena
- DuckDB
- BigQuery, dll

 Kenapa Parquet lebih efisien dibanding CSV?

Aspek	CSV	Parquet
Tipe data	Tidak disimpan (semua string)	Disimpan sebagai schema (typed)
Ukuran file	Besar	Lebih kecil (karena compression)
Akses data	Harus baca semua kolom	Bisa baca kolom tertentu saja
Kecepatan baca	Lambat di dataset besar	Jauh lebih cepat (columnar)
Kompatibilitas	Sangat luas	Terutama untuk big data tools

 Contoh perbandingan penggunaan

Misal kamu punya file transaksi 10 juta baris.

CSV:

- Ukuran bisa 1GB+
- Setiap kali query kolom `total_amount` , tetap baca semua kolom → lambat

Parquet:

- Ukuran bisa cuma 200-300MB (tergantung compression)
- Query hanya kolom `total_amount` → baca cepat karena hanya buka kolom itu saja

 Kapan pakai CSV vs Parquet?

- **CSV** cocok untuk:
 - Data kecil-menengah
 - Kebutuhan human-readable
 - Interoperabilitas tinggi
- **Parquet** cocok untuk:
 - Data besar
 - Dipakai dalam pipeline data lake, data warehouse, Spark, dsb.
 - Kebutuhan query cepat per kolom

Kalau kamu sedang bikin pipeline ETL/data lake, format seperti **Parquet** atau **ORC** sangat direkomendasikan. Tapi untuk transfer data cepat atau debugging manual, CSV masih relevan.

Kalau mau, aku bisa bantuin tunjukkan contoh kode Python baca-tulis CSV dan Parquet juga. Mau?

▼ **Data Lake vs Data Warehouse**

▼ **Data Lake**

Definisi

Data Lake adalah repositori penyimpanan data yang dapat menampung data dalam jumlah besar dalam format aslinya (raw data). Data Lake menyimpan data terstruktur, semi-terstruktur, dan tidak terstruktur tanpa perlu mendefinisikan skema atau struktur terlebih dahulu.

Tujuan

- Menyimpan semua jenis data tanpa batasan format
- Menyediakan fleksibilitas untuk analisis di masa depan

- Mencegah kehilangan data potensial yang mungkin berguna kelak
- Memungkinkan berbagai jenis analisis dari data science hingga machine learning

Manfaat

1. **Fleksibilitas:** Dapat menyimpan data dalam format apapun
2. **Skalabilitas:** Dapat menampung data dalam jumlah sangat besar
3. **Efisiensi biaya:** Biaya penyimpanan per unit data lebih rendah
4. **Tidak ada kehilangan data:** Semua data mentah tersimpan utuh
5. **Future-proof:** Data tersedia untuk kebutuhan analisis yang belum terpikirkan saat ini

Contoh Data Lake

- **Amazon S3:** Object storage yang sering digunakan sebagai data lake
- **Azure Data Lake Storage:** Solusi data lake dari Microsoft
- **Google Cloud Storage:** Platform penyimpanan objek dari Google
- **Hadoop HDFS:** Filesystem terdistribusi yang sering digunakan untuk data lake on-premise

Analogi

Data Lake seperti **danau alami**. Anda dapat menuangkan air (data) dari berbagai sumber, seperti dari sungai, hujan, atau mata air (berbagai sistem) ke dalam danau. Air disimpan dalam bentuk aslinya. Ketika Anda membutuhkan air, Anda harus pergi ke danau, mengambilnya, dan kemudian memprosesnya sesuai kebutuhan (menyaring, mendidihkan, dll.).

▼ Data Warehouse

Definisi

Data Warehouse adalah sistem penyimpanan data yang dirancang khusus untuk analisis dan pelaporan. Data di dalamnya sudah distrukturkan, dibersihkan, dan diorganisasi dengan skema yang sudah ditentukan sebelumnya (schema-on-write).

Tujuan

- Menyediakan data yang terstruktur untuk analisis bisnis
- Mendukung pengambilan keputusan dengan data yang konsisten
- Memungkinkan query dan reporting yang cepat
- Menyediakan "satu versi kebenaran" bagi organisasi

Manfaat

1. **Performa query tinggi:** Dioptimalkan untuk membaca dan menganalisis data
2. **Data berkualitas:** Data sudah dibersihkan dan divalidasi
3. **Konsistensi:** Menyediakan format data yang standar di seluruh organisasi
4. **Kemudahan penggunaan:** Lebih mudah digunakan oleh Business Analyst tanpa keahlian teknis IT yg mendalam
5. **Keamanan:** Kontrol akses yang lebih terperinci dan audit log

Contoh Data Warehouse

- **Google BigQuery:** Data warehouse serverless dari Google
- **Amazon Redshift:** Data warehouse berbasis cloud dari AWS
- **Snowflake:** Data warehouse cloud modern
- **Microsoft Azure Synapse:** Platform analitik terintegrasi dari Microsoft
- **Teradata:** Data warehouse tradisional untuk perusahaan besar

Analogi

Data Warehouse seperti **supermarket**. Data dikumpulkan dari berbagai pemasok (sumber data), kemudian dibersihkan, dikategorikan, dan disusun rapi di rak-rak (tabel) dengan label yang jelas. Pelanggan (pengguna bisnis) dapat dengan mudah menemukan dan mengambil apa yang mereka butuhkan tanpa harus memproses atau membersihkannya lagi.

▼ Perbedaan Data Lake dan Data Warehouse

Aspek	Data Lake	Data Warehouse
Struktur Data	Mentah, tidak diolah (raw)	Terstruktur dan diolah
Schema	Schema-on-read (fleksibel)	Schema-on-write (terdefinisi)
Tujuan	Menyimpan semua data untuk kegunaan yang belum ditentukan	Mendukung analisis bisnis dan reporting
Pengguna	Data scientists, data engineers	Business analysts, BI developers
Ukuran	Sangat besar (petabytes, exabytes)	Lebih kecil (terabytes, petabytes)
Biaya Penyimpanan	Lebih murah	Lebih mahal
Kecepatan Query	Relatif lebih lambat	Cepat
Jenis Data	Terstruktur, semi-terstruktur, tidak terstruktur	Mayoritas terstruktur
Kompleksitas	Memerlukan keahlian teknis tinggi	Lebih mudah digunakan
Contoh Use Case	Machine learning, data discovery, data science	BI reporting, dashboarding, KPI tracking

▼ Penggunaan Dalam Arsitektur Modern

Dalam arsitektur data modern, sering kali Data Lake dan Data Warehouse **digunakan bersama** dalam pola yang disebut "Data Lakehouse":

1. Data mentah masuk ke Data Lake terlebih dahulu
2. Data yang diperlukan untuk analisis bisnis diproses dan dipindahkan ke Data Warehouse
3. Data scientists tetap dapat mengakses data mentah di Data Lake untuk eksplorasi
4. Data engineers membangun pipeline untuk memindahkan data dari Lake ke Warehouse secara berkala

Pendekatan ini menggabungkan fleksibilitas Data Lake dengan kecepatan dan kemudahan penggunaan Data Warehouse.

▼ ETL vs ELT

▼ 1. Definisi

- **ETL (Extract, Transform, Load):**

Proses integrasi data di mana data diekstrak dari sumber heterogen (database, API, file), **ditransformasi** (pembersihan, agregasi, standarisasi) di *staging area*, lalu dimuat ke *data warehouse* atau database tujuan.

- *Contoh Tools:* Informatica, Talend, Microsoft SSIS.

Process Order:

- Extract: Data is extracted from source systems.
- Transform: Data is processed and transformed into a desired format or schema.
- Load: The transformed data is loaded into the destination (e.g., a data warehouse).

- **ELT (Extract, Load, Transform):**

Data diekstrak dari sumber, **langsung dimuat** ke sistem tujuan (data warehouse/data lake), lalu transformasi dilakukan **di dalam sistem tujuan** menggunakan kemampuan pemrosesan sistem tersebut.

- *Contoh Tools:* dbt, Snowflake, BigQuery, Amazon Redshift.

Process Order:

- Extract: Data is extracted from source systems.
- Load: The raw data is loaded into the destination (e.g., a data lake).
- Transform: Transformation happens after loading, within the storage system using its compute resources.

▼ 2. Tujuan

- **ETL:**
 - Memastikan data yang masuk ke sistem tujuan sudah terstruktur, bersih, dan siap pakai.
 - Cocok untuk skenario di mana sistem tujuan memiliki keterbatasan sumber daya (misalnya, *on-premise warehouse*).
- **ELT:**
 - Memanfaatkan kemampuan pemrosesan sistem modern (cloud-based) untuk menangani data mentah dalam skala besar.
 - Ideal untuk data tidak terstruktur (log, sensor IoT) atau *big data*.

▼ 3. Manfaat

ETL:

- **Kepatuhan & Keamanan:** Data sensitif (seperti PII) dapat dienkripsi atau dihapus selama transformasi.
- **Optimasi Storage:** Hanya data yang relevan disimpan di warehouse.
- **Konsistensi:** Data sudah terstandarisasi sebelum dimuat.

ELT:

- **Kecepatan:** Memuat data mentah lebih cepat karena tidak perlu transformasi awal.
- **Fleksibilitas:** Transformasi bisa diubah/diperbarui sesuai kebutuhan analisis.
- **Skalabilitas:** Cocok untuk volume data besar berkat teknologi cloud (elastic computing).

▼ 4. Contoh Implementasi

- **ETL:**

Sebuah bank menggabungkan data transaksi dari cabang-cabang, membersihkan duplikat, memvalidasi format tanggal/nominal, lalu memuatnya ke warehouse untuk laporan keuangan.

 - *Tools:* Talend membersihkan data, lalu memuat ke Oracle Warehouse.
- **ELT:**

Startup e-commerce menyimpan semua klik pengguna (raw JSON) di Google BigQuery, lalu menggunakan SQL/dbt untuk membuat tabel agregasi perilaku pengguna.

 - *Tools:* Apache NiFi mengekstrak data, BigQuery menyimpan dan mentransformasi.

▼ 5. Analogi

- **ETL:**

Seperti restoran yang menyiapkan bahan sebelum disajikan. Sayur dicuci, dipotong, dan dibumbui di dapur (*staging area*), lalu disajikan ke pelanggan (*warehouse*).
- **ELT:**

Seperti supermarket yang menyimpan semua bahan mentah di gudang besar (cloud warehouse), lalu mengolahnya sesuai permintaan pelanggan (misalnya, memotong daging saat ada pesanan).

▼ 6. Perbedaan Utama ETL vs ELT

Aspek	ETL	ELT
Urutan Proses	Transformasi sebelum loading	Transformasi setelah loading
Lokasi Transformasi	Staging area (server terpisah)	Di dalam sistem tujuan (warehouse)
Tipe Data	Terstruktur/skala kecil	Tidak terstruktur/skala besar
Fleksibilitas	Skema rigid (harus ditentukan awal)	Skema fleksibel (schema-on-read)
Kebutuhan Komputasi	Membutuhkan server transformasi	Mengandalkan komputasi cloud
Biaya	Lebih tinggi untuk maintenance ETL	Lebih efisien dengan skalabilitas cloud

▼ 7. Kapan Memilih ETL atau ELT?

- **Pilih ETL Jika:**
 - Data terstruktur dan memerlukan standarisasi ketat.
 - Sistem tujuan (misalnya, warehouse lama) tidak mampu mentransformasi data.
 - Kepatuhan GDPR/CCPA mengharuskan anonimisasi data sebelum penyimpanan.
- **Pilih ELT Jika:**
 - Menggunakan cloud warehouse (BigQuery, Snowflake) dengan kemampuan komputasi tinggi.
 - Butuh analisis cepat atas data mentah (misalnya, IoT/streaming data).
 - Ingin eksplorasi data tanpa skema tetap.

Kesimpulan

ETL dan ELT adalah paradigma integrasi data dengan filosofi berbeda. ETL cocok untuk skenario terstruktur dengan kebutuhan transformasi awal, sementara ELT lebih adaptif untuk era big data dan cloud computing. Pemilihan bergantung pada kebutuhan bisnis, tipe data, dan infrastruktur yang dimiliki.

▼ Introduction to Workflow Orchestration

<https://www.youtube.com/watch?v=ZV6CPZDiJFA>

▼ Analogi Orkestra



Bayangkan sebuah **orkestra** di mana berbagai instrumen musik berbeda harus bermain bersama secara harmonis. Setiap instrumen memiliki peran sendiri, tetapi perlu dimainkan pada waktu yang tepat agar menciptakan harmoni.

Dalam dunia teknologi, **orkestrator** berperan seperti seorang konduktor orkestra. Alih-alih mengoordinasikan instrumen musik, orkestrator mengoordinasikan berbagai **tugas, pipeline, atau layanan mikro (microservices)**. Orkestrator memastikan semua komponen ini bekerja secara harmonis dan terkoordinasi untuk mencapai tujuan yang diinginkan.

▼ Perbedaan antara Orkestrasi dan Otomatisasi

Mungkin Anda bertanya-tanya: "Apa perbedaan antara orkestrasi dan otomatisasi?"

Keduanya memang mirip, namun **orkestrasi** berfokus pada **gambaran besar**:

- Mengatur banyak tugas yang saling bergantung
- Memastikan tugas berjalan dalam urutan yang tepat
- Menangani kegagalan dengan membatalkan tugas terkait
- Memberikan notifikasi kesalahan

Sementara **otomatisasi** sangat baik untuk menjadwalkan tugas tunggal, **orkestrasi** menghubungkan semua tugas tersebut menjadi satu sistem yang terintegrasi dan berfungsi dengan baik.

▼ Definisi Workflow Orchestration

Workflow Orchestration atau Orkestrasi Alur Kerja adalah proses pengorganisasian, pengelolaan, dan otomatisasi alur kerja kompleks, di mana beberapa tugas atau proses dikoordinasikan untuk mencapai hasil tertentu. Orkestrasi melibatkan:

1. Memastikan tugas dijalankan dalam urutan yang benar
2. Mengelola ketergantungan antar tugas
3. Mengalokasikan sumber daya yang diperlukan
4. Menangani kesalahan dan kegagalan

▼ Komponen Utama Workflow Orchestration

1. Koordinasi Tugas

Memastikan tugas dijalankan dalam urutan yang tepat atau secara bersamaan, berdasarkan aturan dan ketergantungan yang telah ditentukan sebelumnya.

2. Otomatisasi

Mengotomatiskan proses yang berulang atau kompleks untuk mengurangi intervensi manual, meningkatkan efisiensi, dan mengurangi kemungkinan kesalahan manusia.

3. Penanganan Kesalahan

Mengelola kesalahan atau kegagalan dalam tugas, sering dengan mekanisme percobaan ulang (retry) atau jalur eksekusi alternatif. Sistem dapat memberitahu tim teknis dan mencegah kerusakan lebih lanjut.

4. Manajemen Sumber Daya

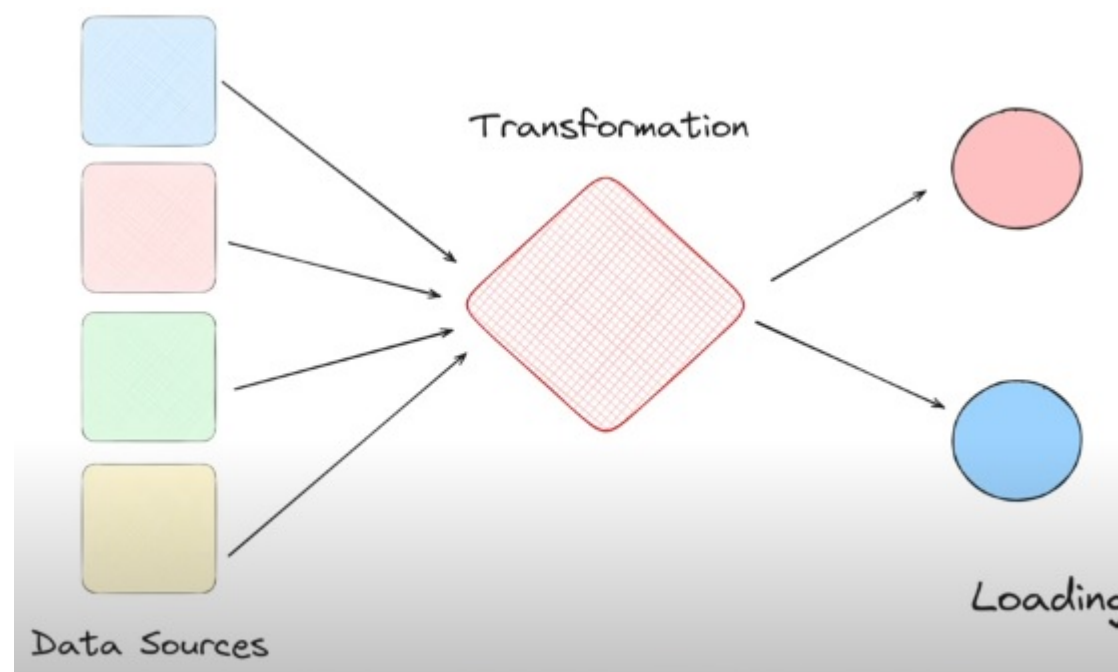
Mengalokasikan sumber daya (seperti daya komputasi, API, atau data) ke tugas sesuai kebutuhan dan memastikan penggunaan yang efisien.

5. Pemantauan dan Pelaporan

Melacak kemajuan alur kerja, mengidentifikasi hambatan, dan menyediakan log atau laporan untuk analisis. Ini memungkinkan transparansi dan perbaikan proses.

▼ Kasus Penggunaan Umum

1. Lingkungan Berbasis Data

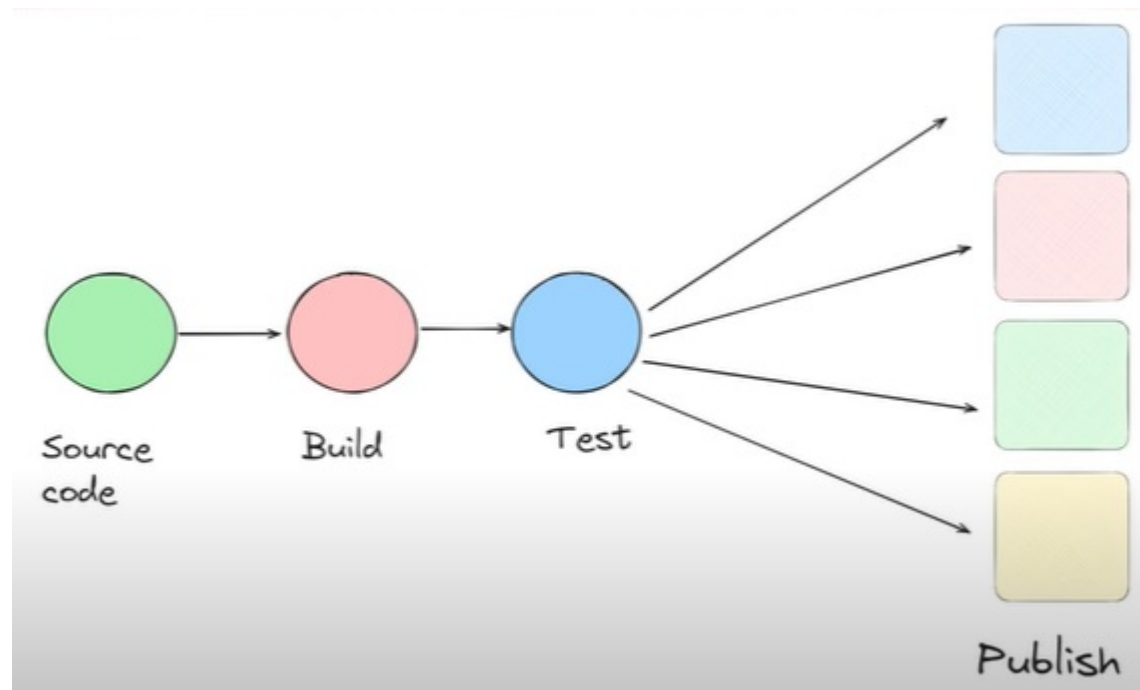


Orkestrator sangat penting dalam proses ETL (Extract, Transform, Load):

- **Extract:** Mengambil data dari berbagai sumber berbeda
- **Transform:** Memproses dan mengubah format data
- **Load:** Memasukkan data ke dalam data warehouse

Orkestrator memastikan setiap langkah berjalan dengan sukses. Jika terjadi kesalahan, orkestrator dapat mencoba ulang langkah tersebut dan mencegah langkah selanjutnya dijalankan. Hal ini menjaga integritas data dan mencegah masalah yang lebih besar.

2. Pipeline CI/CD



Dalam pipeline Continuous Integration/Continuous Deployment (CI/CD), orkestrator sangat berguna untuk:

- Membangun (build) kode aplikasi
- Menjalankan pengujian otomatis
- Mempublikasikan kode ke berbagai lingkungan (development, staging, production)
- Mengkoordinasikan langkah-langkah deployment

Orkestrator memastikan semua langkah terjadi dalam urutan yang benar dan menangani kegagalan dengan tepat.

3. Penyediaan Sumber Daya

Jika infrastruktur Anda berbasis cloud, orkestrator dapat membantu:

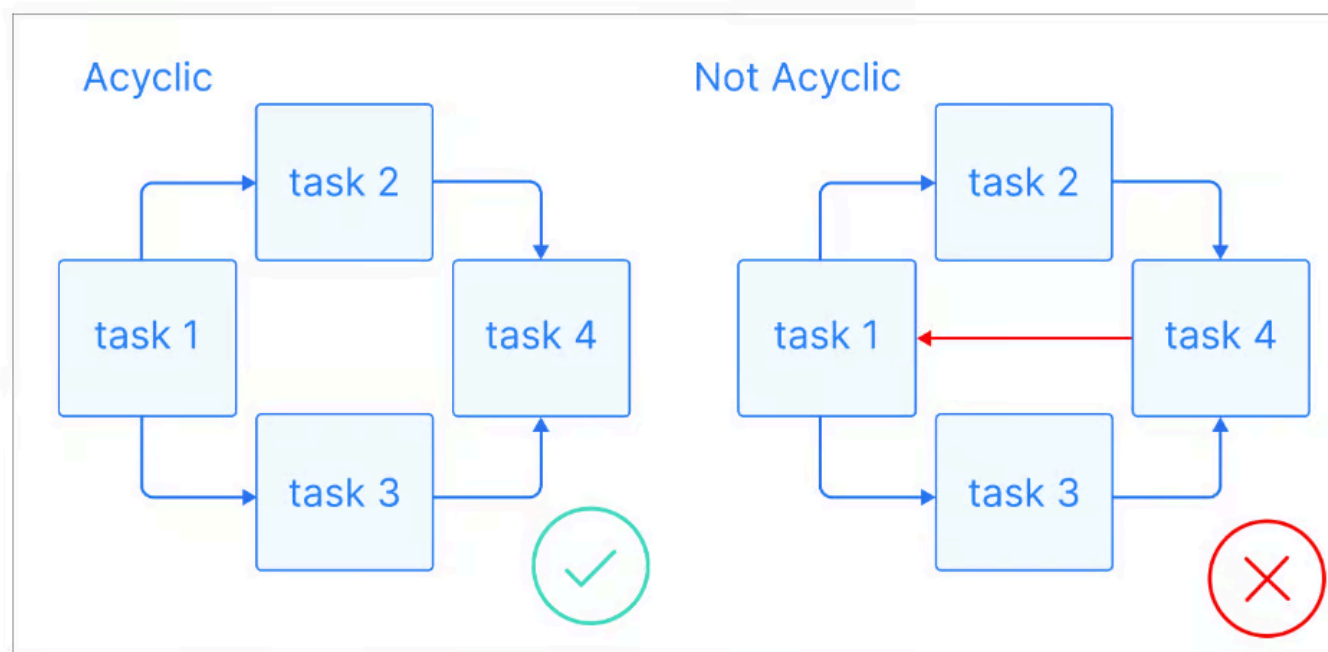
- Menyediakan server virtual
- Mengkonfigurasi jaringan
- Mengatur database
- Mengelola keamanan
- Menerapkan aplikasi

Dengan orkestrator, Anda hanya perlu menekan satu tombol, dan orkestrator akan menyiapkan seluruh lingkungan cloud untuk Anda secara terkoordinasi.

▼ Kesimpulan

Workflow orchestration adalah komponen penting dalam arsitektur sistem modern, terutama di era cloud dan microservices. Dengan mengoordinasikan berbagai tugas, mengelola ketergantungan, dan menangani kesalahan, orkestrator memastikan sistem kompleks berjalan mulus dan efisien. Seperti konduktor dalam orkestra memastikan semua instrumen bermain pada waktu yang tepat untuk menghasilkan musik yang indah, orkestrator dalam teknologi memastikan semua komponen sistem bekerja bersama secara harmonis.

▼ Konsep DAG dalam Workflow Orchestration



Dalam konteks *workflow orchestration*, DAG (Directed Acyclic Graph) adalah representasi visual dan struktural dari alur kerja (workflow) yang ingin Anda otomatisasi dan kelola. Bayangkan DAG sebagai cetak biru atau peta langkah-langkah yang harus diikuti untuk menyelesaikan suatu proses bisnis atau tugas kompleks.

Komponen Utama DAG dalam Konteks Orchestration:

1. **Tasks (Tugas):** Ini adalah unit-unit pekerjaan individual dalam alur kerja Anda. Setiap Task mewakili satu langkah spesifik yang perlu dieksekusi. Contoh Tasks bisa berupa:
 - Mengambil data dari sumber tertentu.
 - Memproses atau mentransformasi data.
 - Menjalankan skrip atau program.
 - Mengirim notifikasi email.
 - Memicu sistem lain.
 - Melakukan validasi data.
2. **Dependencies (Dependensi):** DAG mendefinisikan hubungan dependensi antar Tasks. Ini menentukan urutan eksekusi Tasks. Sebuah Task mungkin perlu menunggu Task lain selesai sebelum dapat dimulai. Dependensi ini direpresentasikan sebagai arah dalam grafik (panah dari Task hulu ke Task hilir). Contoh dependensi:
 - Task B hanya dapat dimulai setelah Task A berhasil diselesaikan.
 - Task C dan Task D dapat berjalan secara paralel setelah Task A selesai.
 - Task E baru dapat dimulai setelah Task B dan Task C selesai.

Peran DAG dalam Workflow Orchestration:

- **Mendefinisikan Alur Eksekusi:** DAG secara eksplisit menentukan urutan Tasks yang harus dijalankan. Ini memastikan bahwa langkah-langkah dalam alur kerja dieksekusi dalam urutan yang benar dan logis.
- **Mengelola Dependensi:** DAG memungkinkan Anda untuk dengan mudah mendefinisikan dan mengelola dependensi antar Tasks. Airflow kemudian akan memastikan bahwa setiap Task hanya dijalankan setelah semua Task yang menjadi dependensinya telah berhasil diselesaikan.
- **Paralelisasi:** DAG memungkinkan Anda untuk mendefinisikan Tasks yang dapat berjalan secara paralel. Ini dapat secara signifikan meningkatkan efisiensi alur kerja dengan memanfaatkan sumber daya komputasi secara lebih optimal.
- **Penjadwalan:** DAG dapat dikonfigurasi untuk berjalan pada jadwal tertentu (misalnya, setiap hari, setiap jam, atau pada waktu tertentu). Ini memungkinkan otomatisasi alur kerja yang berulang.
- **Pemantauan dan Pengelolaan:** Melalui antarmuka web Airflow, Anda dapat memantau status setiap DAG Run dan setiap Task di dalamnya. Anda dapat melihat apakah suatu Task berhasil, gagal, atau masih berjalan. Anda juga dapat melakukan tindakan seperti memicu ulang (*retry*) Task yang gagal atau memicu DAG secara manual.
- **Reproducibility:** DAG mendefinisikan alur kerja secara eksplisit dalam kode (biasanya Python). Ini memastikan bahwa alur kerja dapat direproduksi dan diulang dengan cara yang konsisten.

- **Modularitas:** DAG memungkinkan Anda untuk memecah alur kerja yang kompleks menjadi Tasks yang lebih kecil dan lebih mudah dikelola.

Analogi Sederhana:

Bayangkan Anda ingin membuat kue. Proses pembuatan kue dapat direpresentasikan sebagai DAG:

- **Tasks:** Mencampur bahan kering, mencampur bahan basah, mengocok telur, menggabungkan semua bahan, memanggang, mendinginkan, menghias.
- **Dependencies:** Anda harus mencampur bahan kering dan bahan basah sebelum menggabungkannya. Anda harus memanggang kue sebelum mendinginkannya. Anda tidak dapat menghias kue sebelum mendinginkannya.

DAG akan memastikan bahwa langkah-langkah ini dilakukan dalam urutan yang benar untuk menghasilkan kue yang sempurna.

Manfaat Menggunakan DAG untuk Workflow Orchestration:

- **Otomatisasi:** Mengotomatisasi tugas-tugas manual dan berulang.
- **Efisiensi:** Meningkatkan efisiensi dengan menjalankan tugas secara paralel jika memungkinkan.
- **Reliabilitas:** Memastikan bahwa alur kerja dieksekusi secara konsisten dan dapat diandalkan.
- **Visibilitas:** Memberikan visibilitas yang jelas terhadap status dan kemajuan alur kerja.
- **Kemudahan Pengelolaan:** Memudahkan pengelolaan dan pemantauan alur kerja yang kompleks.

Dengan menggunakan DAG, Airflow menyediakan cara yang kuat dan fleksibel untuk mengelola dan mengotomatisasi berbagai jenis alur kerja, mulai dari pemrosesan data hingga *machine learning pipelines* dan banyak lagi.

▼ Persamaan DAG dengan Flowchart (dalam konteks SWE)

Ya, analogi DAG dengan *flowchart* dalam konteks *Software Engineering* cukup tepat, meskipun ada beberapa perbedaan mendasar yang perlu dipahami. Mari kita telaah persamaannya dan perbedaannya:

Persamaan antara DAG dan Flowchart:

- **Representasi Visual Alur Kerja:** Keduanya menggunakan representasi grafis untuk memvisualisasikan serangkaian langkah atau operasi.
- **Menunjukkan Urutan Eksekusi:** Keduanya menggambarkan urutan di mana langkah-langkah harus dieksekusi.
- **Menggambarkan Dependensi:** Keduanya dapat menunjukkan dependensi antar langkah, di mana suatu langkah mungkin perlu diselesaikan sebelum langkah berikutnya dapat dimulai.
- **Tujuan untuk Pemahaman dan Pengelolaan:** Keduanya bertujuan untuk memberikan pemahaman yang jelas tentang suatu proses dan memfasilitasi pengelolaan proses tersebut.

Perbedaan Utama antara DAG dan Flowchart:

- **Konsep Acyclic (Tanpa Siklus):** Perbedaan paling signifikan adalah bahwa DAG secara eksplisit harus *acyclic* atau tidak boleh memiliki siklus (loop). Ini berarti Anda tidak dapat kembali ke langkah sebelumnya dalam DAG. Dalam *flowchart* program, loop (seperti `for` atau `while`) adalah hal yang umum. Airflow dirancang untuk alur kerja batch yang bergerak maju menuju penyelesaian.
- **Fokus Aplikasi:**
 - **DAG (Airflow):** Secara spesifik dirancang untuk orkestrasi alur kerja (workflow orchestration), terutama yang melibatkan tugas-tugas yang berjalan dalam batch, terjadwal, dan seringkali melibatkan integrasi dengan berbagai sistem eksternal. Penekanannya adalah pada penjadwalan, eksekusi, dan pemantauan tugas-tugas ini.
 - **Flowchart (Software Engineering):** Lebih umum digunakan untuk memvisualisasikan algoritma, logika program, atau alur proses secara umum dalam pengembangan perangkat lunak. Ini bisa mencakup logika internal suatu fungsi atau program, dan tidak terbatas pada alur kerja batch.
- **Lingkup:**
 - **DAG:** Biasanya menggambarkan alur kerja tingkat tinggi yang terdiri dari beberapa tugas yang mungkin berjalan di berbagai sistem atau layanan.
 - **Flowchart:** Bisa menggambarkan logika tingkat tinggi atau logika yang lebih detail di dalam suatu modul atau fungsi program.
- **Eksekusi:**

- **DAG:** Dieksekusi oleh *scheduler* Airflow, yang memastikan bahwa tugas-tugas dijalankan sesuai dengan dependensi yang ditentukan dan jadwal yang dikonfigurasi.
- **Flowchart:** Lebih merupakan representasi statis dari logika program yang kemudian diimplementasikan dalam kode yang dieksekusi oleh *runtime* bahasa pemrograman.

Kesimpulan:

Ya, DAG dalam Airflow memiliki kemiripan konseptual dengan *flowchart* dalam *Software Engineering* karena keduanya merupakan cara untuk memvisualisasikan serangkaian langkah dan dependensinya. Namun, penting untuk diingat bahwa DAG secara khusus dirancang untuk orkestrasi alur kerja batch dan memiliki batasan tidak boleh mengandung siklus (*acyclic*). Sementara *flowchart* lebih merupakan alat visualisasi yang lebih umum untuk berbagai jenis proses dan logika dalam pengembangan perangkat lunak, termasuk yang mungkin mengandung siklus.

Jadi, Anda bisa menganggap DAG sebagai bentuk khusus dari *flowchart* yang dioptimalkan untuk kebutuhan orkestrasi alur kerja dalam konteks seperti pemrosesan data, *data pipelines*, dan tugas-tugas terjadwal lainnya.

▼ Konteks Module 2

We have now separated our pipeline into a **DOWNLOAD** script and a **INGEST** script.

In this lesson we will create a more complex pipeline:

```
(web)
↓
DOWNLOAD
↓
(csv)
↓
PARQUETIZE
↓
(parquet) -----> UPLOAD TO S3
↓
UPLOAD TO GCS
↓
(parquet in GCS)
↓
UPLOAD TO BIGQUERY
↓
(table in BQ)
```

Parquet is a columnar storage datafile format which is more efficient than CSV.

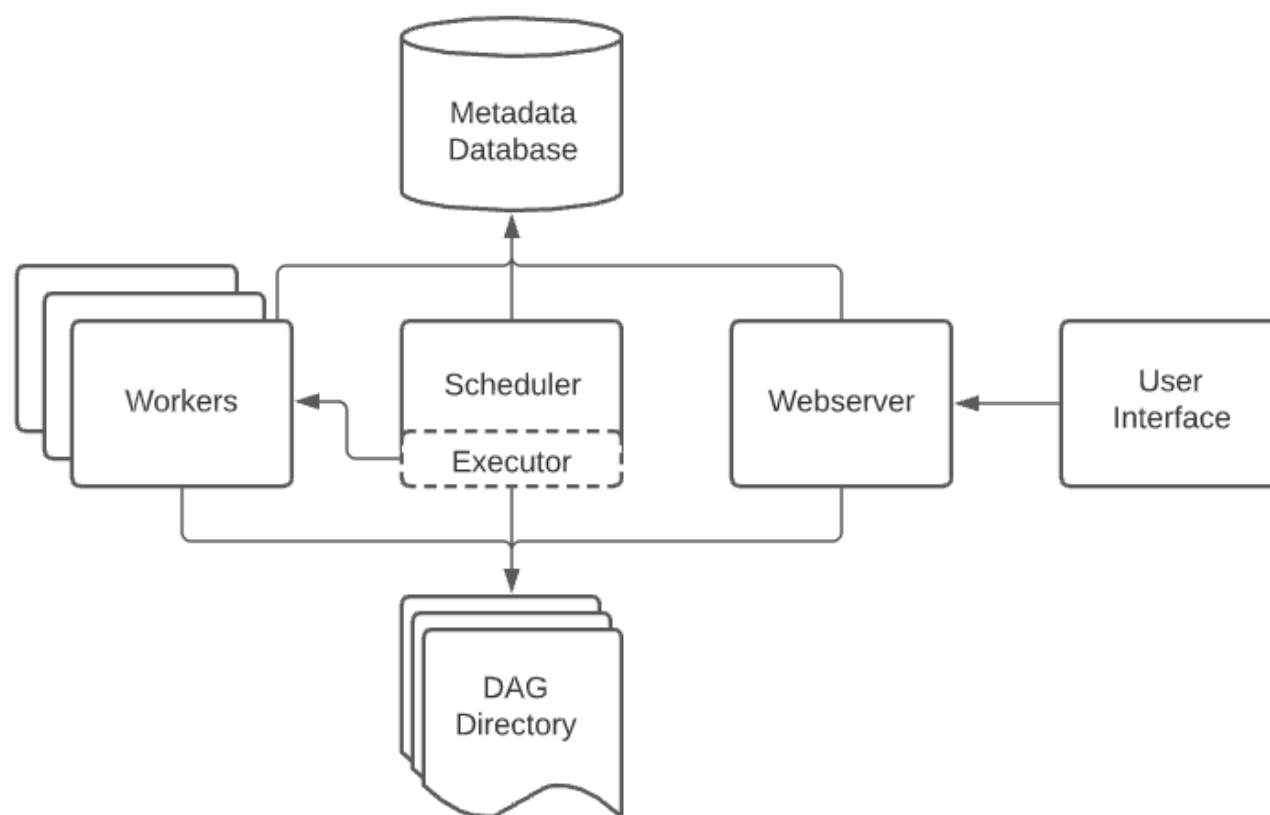
This **Data Workflow** has more steps and even branches. This type of workflow is often called a **Directed Acyclic Graph** (DAG) because it lacks any loops and the data flow is well defined.

The steps in capital letters are our **jobs** and the objects in between are the jobs' outputs, which behave as **dependencies** for other jobs. Each job may have its own set of **parameters** and there may also be global parameters which are the same for all of the jobs.

A **Workflow Orchestration Tool** allows us to define data workflows and parametrize them; it also provides additional tools such as **history and logging**.

The tool we will focus on in this course is **Apache Airflow**, but there are many others such as Luigi, Prefect, Argo, etc.

▼ Arsitektur Apache Airflow



<https://airflow.apache.org/docs/apache-airflow/2.4.1/concepts/overview.html>

▼ Pengantar

Apache Airflow® adalah platform sumber terbuka (open-source) yang digunakan untuk mengembangkan, menjadwalkan, dan memantau alur kerja (workflows) yang berorientasi pada pemrosesan batch. Airflow memungkinkan Anda untuk membangun dan menjalankan alur kerja yang direpresentasikan sebagai DAG (Directed Acyclic Graph). Setiap DAG terdiri dari unit-unit pekerjaan individual yang disebut Tasks, yang diatur dengan mempertimbangkan dependensi dan aliran data.

Sebuah DAG mendefinisikan dependensi antar Tasks, yang menentukan urutan eksekusi Tasks. Tasks sendiri menjelaskan tindakan yang perlu dilakukan, seperti mengambil data, menjalankan analisis, atau memicu sistem lain.

▼ Bab 1: Komponen Utama Airflow (Airflow Components)

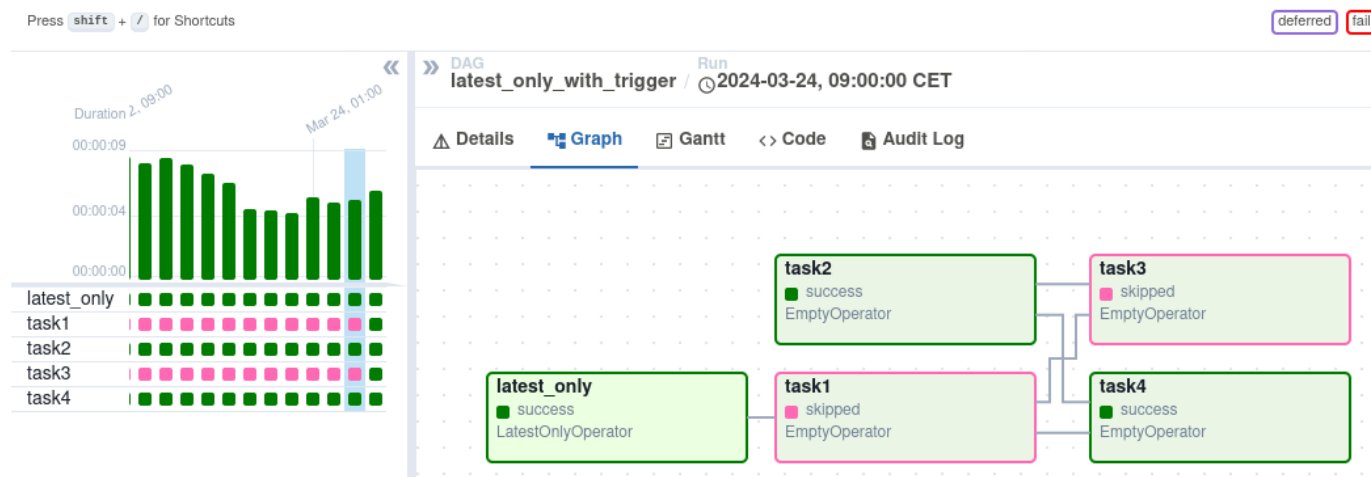
Instalasi Airflow yang minimal terdiri dari komponen-komponen berikut:

- **Scheduler:** Komponen ini bertanggung jawab untuk memicu alur kerja terjadwal dan mengirimkan Tasks ke **executor** untuk dijalankan. Penting untuk dicatat bahwa **executor** bukanlah komponen terpisah, melainkan properti konfigurasi dari **scheduler** yang berjalan dalam proses **scheduler**. Airflow menyediakan beberapa **executor** bawaan, dan pengguna juga dapat membuat **executor** kustom.
- **Webserver:** Menyediakan antarmuka pengguna (user interface) yang mudah digunakan untuk memeriksa, memicu, dan melakukan *debugging* terhadap perilaku DAG dan Tasks.
- **Folder DAG:** Direktori yang berisi file-file Python yang dibaca oleh **scheduler** untuk menentukan Tasks mana yang perlu dijalankan dan kapan.
- **Metadata Database:** Basis data yang digunakan oleh komponen-komponen Airflow untuk menyimpan status alur kerja dan Tasks. Konfigurasi basis data metadata sangat penting agar Airflow dapat berfungsi.

Selain komponen utama, terdapat juga komponen opsional:

- **Worker:** Komponen opsional yang bertugas mengeksekusi Tasks yang diberikan oleh **scheduler**. Dalam instalasi dasar, **worker** mungkin merupakan bagian dari proses **scheduler**. Namun, pada konfigurasi yang lebih kompleks seperti menggunakan **CeleryExecutor** atau **KubernetesExecutor**, **worker** dapat dijalankan sebagai proses jangka panjang atau sebagai POD terpisah.
- **Folder Plugin:** Direktori opsional yang berisi *plugin*. *Plugin* adalah cara untuk memperluas fungsionalitas Airflow (mirip dengan paket yang diinstal). *Plugin* dibaca oleh **scheduler**, **dag processor**, **triggerer**, dan **webserver**.

▼ Bab 2: DAG (Directed Acyclic Graph)



DAG adalah konsep inti dalam Airflow. DAG mengumpulkan Tasks bersama-sama, diorganisir dengan dependensi dan hubungan untuk menentukan bagaimana Tasks tersebut harus dijalankan.

Contoh DAG sederhana:



Contoh di atas mendefinisikan empat Tasks (A, B, C, dan D) dan menentukan urutan eksekusinya serta dependensi antar Tasks. DAG juga menentukan frekuensi eksekusi, misalnya "setiap 5 menit mulai besok" atau "setiap hari sejak 1 Januari 2020".

DAG tidak peduli dengan apa yang terjadi di dalam Tasks; fokus utamanya adalah bagaimana mengeksekusi Tasks tersebut, termasuk urutan eksekusi, jumlah percobaan ulang (*retries*), dan pengaturan *timeout*.

Sub-bab 2.1: Mendeklarasikan DAG (Declaring a DAG)

Ada tiga cara utama untuk mendeklarasikan DAG:

1. Menggunakan `with statement` :

Python

```
import datetime

from airflow import DAG
from airflow.operators.empty import EmptyOperator

with DAG(
    dag_id="my_dag_name",
    start_date=datetime.datetime(2021, 1, 1),
    schedule="@daily",
):
    EmptyOperator(task_id="task")
```

2. Menggunakan konstruktor standar dengan melewati objek DAG ke operator:

Python

```
import datetime

from airflow import DAG
from airflow.operators.empty import EmptyOperator

my_dag = DAG(
    dag_id="my_dag_name",
    start_date=datetime.datetime(2021, 1, 1),
    schedule="@daily",
)

EmptyOperator(task_id="task", dag=my_dag)
```

3. Menggunakan dekorator `@dag` untuk mengubah fungsi menjadi generator DAG:

Python

```
import datetime

from airflow.decorators import dag
from airflow.operators.empty import EmptyOperator

@dag(start_date=datetime.datetime(2021, 1, 1), schedule="@daily")
def generate_dag():
    EmptyOperator(task_id="task")

generate_dag()
```

Sub-bab 2.2: Memuat DAG (Loading DAGs)

Airflow memuat DAG dari file-file sumber Python yang terletak di dalam direktori `DAG_FOLDER` yang telah dikonfigurasi. Airflow akan mengeksekusi setiap file dan memuat objek DAG yang ditemukan di dalamnya. Ini memungkinkan Anda untuk mendefinisikan beberapa DAG dalam satu file Python atau bahkan membagi satu DAG yang kompleks menjadi beberapa file menggunakan `import`.

Sub-bab 2.3: Menjalankan DAG (Running DAGs)

Ada dua cara utama untuk menjalankan DAG:

- 1. **Memicu secara manual (Triggering manually):** Melalui antarmuka web (web UI) atau secara programatik melalui API. Antarmuka web Airflow memungkinkan pengguna untuk memicu eksekusi DAG secara manual. Ini sering digunakan untuk menjalankan DAG di luar jadwal yang ditentukan, misalnya untuk tugas *ad-hoc* atau untuk pengujian. Di web UI, Anda dapat mengklik tombol "Trigger Dag" di samping DAG yang ingin dieksekusi.
- 2. **Menjadwalkan (Scheduling):** Airflow memungkinkan Anda untuk menjadwalkan DAG untuk berjalan secara otomatis pada interval waktu tertentu. Ini adalah cara paling umum untuk menjalankan DAG, karena memungkinkan eksekusi rutin tanpa intervensi manual. Jadwal didefinisikan menggunakan ekspresi cron atau jadwal yang telah ditentukan seperti `@daily`, `@hourly`, dll., dalam definisi DAG. **Scheduler** Airflow akan memantau DAG dan memicu Tasks berdasarkan jadwal yang ditentukan.

Sub-bab 2.4: DAG Runs

Browse Admin Docs About 15:50 UTC					
Schedule	Owner	Recent Tasks	Last Run	DAG Runs	Links
47 ****	Airflow	1 1	2018-02-18 01:02	34	
25 0 ***	airflow	3 upstream_failed	2018-02-18 00:25	6 1	
0 ****	airflow	3	2018-02-19 14:00	235 4	
0 ****	airflow	3	2018-02-19 14:00	235 1	
0 ****	airflow	3	2018-02-19 14:00	136 2	
0 4 ***	airflow	2	2018-02-18 04:00	7	
0 0 ** 1	airflow	1	2018-02-12 00:00	1 1	
22 ****	airflow	3	2018-02-19 14:22	89 3	
19 ****	airflow	3	2018-02-19 14:19	88 4	
29/30 ****	airflow	6	2018-02-19 14:59	331 31	
30 ****	airflow	5	2018-02-19 14:30	117 3	

Sebuah **DAG Run** adalah objek yang merepresentasikan instansiasi DAG pada waktu tertentu. Setiap kali DAG dieksekusi, sebuah **DAG Run** dibuat dan semua Tasks di dalamnya dijalankan. Status **DAG Run** bergantung pada

status Tasks. Setiap **DAG Run** berjalan secara terpisah satu sama lain, yang berarti Anda dapat memiliki banyak eksekusi DAG secara bersamaan.

Status **DAG Run** ditentukan setelah eksekusi DAG selesai. Ada dua status terminal yang mungkin untuk **DAG Run**:

- **success**: Jika semua status *leaf nodes* (node terakhir dalam DAG) adalah `success` atau `skipped`.
- **failed**: Jika ada salah satu status *leaf nodes* adalah `failed` atau `upstream_failed`.

▼ Bab 3: Tasks

Sebuah Task adalah unit eksekusi dasar dalam Airflow. Tasks diatur dalam DAG, dan dependensi *upstream* dan *downstream* ditetapkan di antara mereka untuk menentukan urutan eksekusi.

Ada tiga jenis dasar Task:

- **Operators**: Template Tasks yang telah ditentukan sebelumnya yang dapat Anda rangkai dengan cepat untuk membangun sebagian besar DAG Anda.
- **Sensors**: Subkelas khusus dari Operators yang sepenuhnya berfokus pada menunggu terjadinya peristiwa eksternal (untuk IoT).
- **TaskFlow-decorated** `@task`: Fungsi Python kustom yang dikemas sebagai Task.

Sub-bab 3.1: Operators

Sebuah Operator secara konseptual adalah template untuk Task yang telah ditentukan sebelumnya, yang dapat Anda definisikan secara deklaratif di dalam DAG Anda:

Python

```
from airflow import DAG
from airflow.operators.http import HttpOperator
from airflow.operators.email import EmailOperator
from datetime import datetime

with DAG("my-dag", start_date=datetime(2023, 1, 1)) as dag:
    ping = HttpOperator(endpoint="http://example.com/update/", task_id="ping_endpoint")
    email = EmailOperator(to="admin@example.com", subject="Update complete", task_id="send_email")

    ping >> email
```

Airflow memiliki banyak sekali Operators yang tersedia, baik yang merupakan bagian dari inti (core) maupun yang disediakan oleh *providers* yang telah diinstal sebelumnya. Beberapa Operators populer dari *core* meliputi:

- **BashOperator**: Mengeksekusi perintah bash.
- **PythonOperator**: Memanggil fungsi Python arbitrer.
- **EmailOperator**: Mengirim email.

▼ Bab 4: Executor

Executors adalah mekanisme yang digunakan untuk menjalankan *task instances*. **Executors** diatur melalui opsi `executor` di bagian `[core]` dari file konfigurasi Airflow.

Ada dua jenis utama **executors**: yang menjalankan Tasks secara lokal (di dalam proses **scheduler**) dan yang menjalankan Tasks secara *remote* (biasanya melalui kumpulan **workers**).

Sub-bab 4.1: Local Executors

Tasks Airflow dijalankan secara lokal di dalam proses **scheduler**.

- **Kelebihan (Pros)**: Sangat mudah digunakan, cepat, latensi sangat rendah, dan sedikit persyaratan untuk konfigurasi.
- **Kekurangan (Cons)**: Kemampuan terbatas dan berbagi sumber daya dengan **scheduler** Airflow.
- **Contoh (Examples)**: `LocalExecutor`, `SequentialExecutor`.

Sub-bab 4.2: Remote Executors

Remote executors dapat dibagi lagi menjadi dua kategori:

1. **Queued/Batch Executors:** Tasks Airflow dikirim ke antrian pusat di mana **workers** *remote* mengambil Tasks untuk dieksekusi. Seringkali **workers** bersifat persisten dan menjalankan beberapa Tasks sekaligus.
 - **Kelebihan (Pros):** Lebih *robust* karena memisahkan **workers** dari proses **scheduler**. **Workers** dapat berupa *host* besar yang dapat memproses banyak Tasks (seringkali secara paralel) yang hemat biaya. Latensi bisa relatif rendah karena **workers** dapat diprovisikan untuk selalu berjalan dan segera mengambil Tasks dari antrian.
 - **Kekurangan (Cons):** **Workers** bersama dapat menimbulkan masalah "tetangga berisik" di mana Tasks bersaing untuk sumber daya pada *host* yang sama atau bersaing untuk konfigurasi lingkungan/sistem. Mereka juga bisa mahal jika beban kerja Anda tidak konstan, Anda mungkin memiliki **workers** yang menganggur, sumber daya yang terlalu besar, atau Anda harus mengelola *scaling* naik dan turun.
 - **Contoh (Examples):** `CeleryExecutor` , `BatchExecutor` .
2. **Containerized Executors:** Tasks Airflow dieksekusi secara *ad hoc* di dalam *containers/pods*. Setiap Task diisolasi dalam lingkungan *containerized* sendiri yang di-deploy saat Task Airflow diantrikan.
 - **Kelebihan (Pros):** Setiap Task Airflow diisolasi ke dalam satu *container* sehingga tidak ada masalah "tetangga berisik". Lingkungan eksekusi dapat disesuaikan untuk Tasks tertentu (pustaka sistem, *binaries*, dependensi, jumlah sumber daya, dll.). Hemat biaya karena **workers** hanya aktif selama durasi Task.
 - **Kekurangan (Cons):** Ada latensi saat startup karena *container* atau *pod* perlu di-deploy sebelum Task dapat dimulai. Bisa mahal jika Anda menjalankan banyak Tasks pendek/kecil. Tidak ada **workers** untuk dikelola, tetapi Anda harus mengelola sesuatu seperti kluster Kubernetes.
 - **Contoh (Examples):** `KubernetesExecutor` , `EcsExecutor` .

Referensi

Untuk informasi lebih lanjut, silakan kunjungi dokumentasi resmi:

<https://airflow.apache.org/docs/apache-airflow/stable/core-concepts/index.html>¹

▼ Bab 5: Control Flow

Control Flow dalam konteks Airflow mengacu pada urutan di mana Tasks dalam sebuah DAG dieksekusi. Ini adalah bagaimana Airflow menentukan jalur yang harus diambil oleh sebuah *DAG Run* dari awal hingga akhir.

Sub-bab 5.1: Dasar-dasar Control Flow

DAG dirancang untuk dijalankan berkali-kali (reusable), dan beberapa *runs* dari DAG yang sama dapat terjadi secara paralel. Setiap *DAG Run* diparameterisasi, selalu menyertakan interval waktu yang menjadi fokus eksekusinya (*data interval*), namun juga dapat memiliki parameter opsional lainnya.

Alur *Control Flow* dalam DAG utamanya ditentukan oleh dependensi yang dideklarasikan antar Tasks. Secara *default*, sebuah Task akan menunggu semua Tasks yang berada di *upstream* (yang menjadi dependensinya) berhasil diselesaikan sebelum Task tersebut dijalankan.

Sub-bab 5.2: Mendeklarasikan Dependensi untuk Control Flow

Anda mendeklarasikan dependensi antar Tasks menggunakan operator `>>` (downstream) dan `<<` (upstream), atau menggunakan metode `set_upstream` dan `set_downstream` .

- **Menggunakan Operator `>>` dan `<<` :**
Ini adalah cara yang direkomendasikan dan paling umum:

Python

```
first_task >> [second_task, third_task] # second_task dan third_task bergantung pada first_task
fourth_task << third_task               # fourth_task bergantung pada third_task
```

- **Menggunakan Metode `set_upstream` dan `set_downstream` :**
Ini adalah cara yang lebih eksplisit:

Python

```
first_task.set_downstream([second_task, third_task]) # Menetapkan second_task dan third_task sebagai downs
fourth_task.set_upstream(third_task)                # Menetapkan third_task sebagai upstream dari fourth_task
```

Dependensi inilah yang membentuk "sisi" (*edges*) dari grafik, dan bagaimana Airflow mengetahui urutan eksekusi Tasks Anda.

Sub-bab 5.3: Kustomisasi Control Flow

Meskipun perilaku *default* adalah menunggu semua *upstream* berhasil, Airflow menyediakan fitur untuk menyesuaikan *Control Flow* berdasarkan kondisi atau logika tertentu:

- **Branching:** Memungkinkan alur kerja bercabang berdasarkan hasil Task tertentu (misalnya, jalankan Task A jika kondisi X terpenuhi, atau Task B jika tidak).
- **LatestOnly:** Memastikan bahwa hanya *DAG Run* terbaru yang terus berjalan ketika ada *DAG Run* yang tumpang tindih.
- **Trigger Rules:** Mengontrol kapan sebuah Task harus dieksekusi berdasarkan status *upstream* Tasks-nya (misalnya, jalankan Task ini bahkan jika salah satu *upstream* gagal, atau hanya jika setidaknya satu *upstream* berhasil).

Sub-bab 5.4: Passing Data Antar Tasks

Untuk melewati data antar Tasks (yang memengaruhi *Control Flow* berdasarkan data):

- **XComs ("Cross-communications"):** Sistem bawaan Airflow untuk Tasks mendorong (*push*) dan menarik (*pull*) sejumlah kecil metadata.
- **Sistem Penyimpanan Eksternal:** Mengunggah dan mengunduh file besar dari layanan penyimpanan (cloud storage seperti S3, GCS, atau *share* jaringan).
- **TaskFlow API:** Secara otomatis meneruskan data antar Tasks melalui *implicit XComs*, menyederhanakan penulisan Python functions sebagai Tasks.

Perlu dicatat bahwa Airflow mengirimkan Tasks untuk dijalankan di *Workers* yang tersedia, sehingga tidak ada jaminan semua Tasks dalam DAG Anda akan berjalan pada *Worker* atau mesin yang sama.

Sub-bab 5.5: Mengelola Kompleksitas DAG

Ketika DAG menjadi sangat kompleks, Airflow menyediakan mekanisme untuk mengelolanya:

- **SubDAGs:** Memungkinkan Anda membuat DAG "yang dapat digunakan kembali" (*reusable*) yang dapat disematkan ke dalam DAG lain. (Catatan: Penggunaan SubDAGs cenderung berkurang dalam versi Airflow yang lebih baru demi TaskGroups).
- **TaskGroups:** Memungkinkan Anda mengelompokkan Tasks secara visual di antarmuka pengguna (UI), membuat DAG yang kompleks lebih mudah dipahami dan dinavigasi.

Sub-bab 5.6: Fitur Pendukung Control Flow

- **Connections & Hooks:** Fitur untuk mengonfigurasi akses ke sumber daya eksternal (seperti basis data, layanan cloud) di satu tempat, memudahkan Tasks yang berbeda untuk berinteraksi dengan sumber daya tersebut.
- **Pools:** Fitur untuk membatasi konkurensi (jumlah Task yang dapat berjalan bersamaan) untuk sekumpulan Tasks tertentu, membantu mengelola beban pada sistem eksternal atau sumber daya bersama.

▼ Bab 6: Task Dependencies

Task Dependencies adalah cara Anda mendefinisikan hubungan antara Tasks dalam DAG, secara spesifik menentukan Task mana yang harus diselesaikan sebelum Task lain dapat dimulai. Ini adalah fondasi dari *Control Flow* dalam Airflow.

Sub-bab 6.1: Pengertian Upstream dan Downstream

- **Upstream:** Task yang harus diselesaikan *sebelum* Task saat ini dapat dimulai.
- **Downstream:** Task yang *bergantung* pada penyelesaian Task saat ini untuk dapat dimulai.

Mendeklarasikan dependensi inilah yang membentuk struktur DAG (sisi terarah dari grafik).

Sub-bab 6.2: Cara Mendeklarasikan Dependensi Individu

Ada dua cara utama untuk mendeklarasikan dependensi satu per satu:

1. Menggunakan Operator `>>` dan `<<` (Direkomendasikan):

- Operator `>>` berarti "adalah *downstream* dari" atau "datang setelah".
- Operator `<<` berarti "adalah *upstream* dari" atau "datang sebelum".

Python

```
first_task >> [second_task, third_task] # first_task adalah upstream dari second_task dan third_task
third_task << fourth_task             # fourth_task adalah upstream dari third_task
```

Dalam contoh pertama, `second_task` dan `third_task` hanya akan berjalan setelah `first_task` selesai. Dalam contoh kedua, `third_task` hanya akan berjalan setelah `fourth_task` selesai.

2. Menggunakan Metode `set_upstream` dan `set_downstream`:

Ini adalah cara yang lebih eksplisit dan kadang digunakan untuk fleksibilitas.

Python

```
first_task.set_downstream(second_task, third_task) # set second_task dan third_task sebagai downstream dari first_task
third_task.set_upstream(fourth_task)              # set fourth_task sebagai upstream dari third_task
```

Sub-bab 6.3: Cara Mendeklarasikan Dependensi Kompleks

Airflow menyediakan cara pintas untuk mendeklarasikan dependensi yang lebih kompleks antar daftar Tasks:

1. `cross_downstream` : Digunakan untuk membuat semua Task dalam satu daftar menjadi *downstream* dari semua Task dalam daftar lain.

Python

```
from airflow.models.baseoperator import cross_downstream

# Menggantikan:
# [op1, op2] >> op3
# [op1, op2] >> op4
cross_downstream([op1, op2], [op3, op4])
# Ini akan membuat dependensi: op1 >> op3, op1 >> op4, op2 >> op3, op2 >> op4
```

2. `chain` : Digunakan untuk merangkai dependensi secara berurutan atau berpasangan.

- **Urutan Sederhana:**

Python

```
from airflow.models.baseoperator import chain

# Menggantikan: op1 >> op2 >> op3 >> op4
chain(op1, op2, op3, op4)
```

- **Rantai Dinamis:**

Python

```
# Membuat rantai op1 >> op2 >> op3 >> op4 >> op5
chain(*[EmptyOperator(task_id='op' + str(i)) for i in range(1, 6)])
```

- **Rantai Berpasangan (*Pairwise*) untuk Daftar Berukuran Sama:** Ini berbeda dengan `cross_downstream` . Ini membuat Task di posisi `i` dalam daftar pertama menjadi *upstream* dari Task di posisi `i` dalam daftar kedua, dan seterusnya.

Python

```
from airflow.models.baseoperator import chain
```

```
# Menggantikan:
# op1 >> op2 >> op4 >> op6
# op1 >> op3 >> op5 >> op6
# (Perhatikan bahwa op1 adalah upstream dari kedua elemen pertama di daftar kedua)
chain(op1, [op2, op3], [op4, op5], op6)
# Ini akan membuat dependensi: op1 >> op2, op1 >> op3, op2 >> op4, op3 >> op5, op4 >> op6, op5 >> op6
```

Memahami cara mendeklarasikan dan mengelola *Task Dependencies* adalah kunci untuk membangun DAG yang berfungsi dengan benar dan mencerminkan alur kerja yang ingin Anda orkestrasi.

▼ Penjelasan Lebih Lanjut Komponen Airflow

Mari kita jelaskan satu per satu, lalu gunakan analogi.

1. Scheduler:

- **Peran Utama:** Otak dari Airflow. Tugasnya adalah memantau folder DAG, mendeteksi perubahan pada file DAG, dan menentukan *kapan* suatu DAG (atau *Task* di dalamnya) harus dijalankan berdasarkan jadwal (*schedule*) atau pemicu manual (*manual trigger*) dan dependensi antar Tasks.
- **Apa yang Dilakukan:** Ketika tiba waktunya untuk menjalankan sebuah DAG atau Task, Scheduler yang memutuskan bahwa "Task X di DAG Y sekarang siap untuk dieksekusi". Kemudian, Scheduler akan memberitahu Executor untuk menjalankan Task tersebut.

2. Executor:

- **Peran Utama:** Mekanisme atau "sistem" yang *sebenarnya menjalankan* Task. Executor menerima instruksi dari Scheduler dan bertanggung jawab untuk memastikan Task tersebut dieksekusi di suatu tempat.
- **Apa yang Dilakukan:** Executor bukanlah orang atau mesin *itu sendiri* yang melakukan pekerjaan, melainkan *cara* pekerjaan itu diserahkan dan dijalankan. Ada berbagai jenis Executor (misalnya, `LocalExecutor`, `CeleryExecutor`, `KubernetesExecutor`), dan jenis Executor yang Anda pilih menentukan bagaimana Task akan didistribusikan dan dijalankan. Dalam konfigurasi Airflow, Executor adalah properti dari Scheduler.

3. Worker:

- **Peran Utama:** Entitas yang *melakukan pekerjaan sebenarnya* dari sebuah Task. Worker adalah "mesin" atau "proses" yang mengeksekusi kode Task (misalnya, menjalankan skrip Python atau perintah Bash).
- **Apa yang Dilakukan:** Worker *tidak selalu* merupakan komponen terpisah. Keberadaan Worker sebagai komponen terpisah hanya ada jika Anda menggunakan *remote executor* seperti `CeleryExecutor` atau `KubernetesExecutor`. Pada *local executors* seperti `LocalExecutor` atau `SequentialExecutor`, fungsi Worker dijalankan langsung di dalam proses Scheduler/Executor itu sendiri.

Analogi: Proyek Konstruksi (Pembangunan Rumah)

Bayangkan Anda adalah manajer proyek yang ingin membangun sebuah rumah (Workflow / DAG). Proses membangun rumah terdiri dari banyak tugas kecil (Tasks) seperti menggali pondasi, memasang bata, membangun atap, mengecat dinding, dll., yang harus dilakukan dalam urutan tertentu (Dependencies).

1. Project Manager (Scheduler):

- Anda, sebagai Project Manager, memegang *blueprint* rumah (DAG file).
- Anda melihat jadwal pembangunan (*schedule*).
- Anda terus memeriksa apakah tahapan sebelumnya (*upstream tasks*) sudah selesai.
- Setelah "menggali pondasi" selesai, Anda memutuskan bahwa "memasang bata" sekarang siap untuk dimulai. Anda memberitahu tim Anda (melalui Foreman atau sistem penugasan) bahwa tugas "memasang bata" perlu dijalankan.

2. Sistem Penugasan/Foreman (Executor):

- Anda, sebagai Project Manager, memiliki cara untuk mendelegasikan tugas kepada para pekerja. Ini bisa sesederhana memberi tahu langsung kepada tukang (misalnya, saat proyek kecil), atau melalui seorang mandor (Foreman), atau melalui sistem daftar tugas di papan tulis. Ini adalah **Sistem Penugasan/Foreman** Anda, yaitu **Executor**.
- Executor menerima instruksi dari Anda (Scheduler) untuk "jalankan tugas memasang bata".

- Executor bertanggung jawab untuk memastikan bahwa tugas "memasang bata" diberikan kepada seseorang (Worker) untuk dieksekusi.

3. Tukang/Pekerja Konstruksi (Worker):

- Mereka adalah orang-orang (atau mesin) yang *benar-benar* melakukan pekerjaan fisik di lokasi konstruksi. Mereka adalah **Tukang/Pekerja Konstruksi** Anda, yaitu **Worker**.
- Mereka menerima tugas "memasang bata" dari Sistem Penugasan/Foreman (Executor).
- Mereka melakukan pekerjaan memasang bata sampai selesai.

Bagaimana Analogi Bekerja dalam Konfigurasi Berbeda:

- **Konfigurasi Sederhana (LocalExecutor / SequentialExecutor):**
 - Anda (Project Manager / Scheduler) memutuskan kapan "memasang bata" siap.
 - **Anda sendiri** (karena proyeknya kecil, Anda merangkap) mengambil alat dan **melakukan sendiri** tugas "memasang bata".
 - Dalam kasus ini, **Scheduler, Executor, dan Worker semuanya adalah proses yang sama (Anda)**. Scheduler memutuskan, Executor (sistem penugasan diri) adalah diri Anda, dan Worker (pelaksana tugas) juga diri Anda.
- **Konfigurasi Kompleks (CeleryExecutor / KubernetesExecutor):**
 - Anda (Project Manager / Scheduler) memutuskan kapan "memasang bata" siap.
 - Anda memberitahu Foreman (Executor) untuk menugaskan tugas ini.
 - Foreman (Executor) melihat ada beberapa Tukang/Pekerja Konstruksi (Workers) yang tersedia di lokasi.
 - Foreman menugaskan tugas "memasang bata" kepada salah satu Tukang yang tersedia.
 - Tukang yang ditugaskan (Worker) pergi ke lokasi yang ditentukan dan melakukan pekerjaan "memasang bata".
 - Dalam kasus ini, **Scheduler** adalah Anda yang merencanakan dan memutuskan, **Executor** adalah sistem penugasan (Foreman/papan tugas) yang mendistribusikan, dan **Workers** adalah para tukang yang terpisah dan independen yang menerima tugas dari Executor.

Ringkasan Peran:

- **Scheduler:** *Apa dan Kapan* Task siap dijalankan.
- **Executor:** *Bagaimana* Task didistribusikan untuk dijalankan.
- **Worker:** *Siapa atau Apa* yang melakukan eksekusi Task, seringkali merupakan entitas terpisah pada konfigurasi skala besar.

Semoga analogi proyek konstruksi ini membantu memperjelas peran masing-masing komponen!

▼ Konsep External Table

Sederhananya, *external table* di BigQuery adalah sebuah **meta-definisi** atau **pointer** ke data yang sebenarnya tersimpan di luar BigQuery, biasanya di Google Cloud Storage (GCS). Ketika Anda membuat *external table*, Anda tidak memindahkan atau menyalin data ke dalam penyimpanan BigQuery. Anda hanya memberi tahu BigQuery:

1. **Lokasi Data:** Di mana file data Anda berada di GCS (path ke bucket dan file).
2. **Format Data:** Bagaimana data tersebut terstruktur (misalnya, CSV, JSON, Avro, Parquet, ORC).
3. **Skema (opsional tapi disarankan):** Struktur kolom dan tipe datanya. BigQuery dapat mencoba mendeteksi skema secara otomatis untuk beberapa format, tetapi mendefinisikannya secara eksplisit lebih baik untuk keandalan.

Setelah *external table* dibuat, Anda dapat melakukan *query* SQL terhadapnya seolah-olah itu adalah tabel BigQuery biasa. Namun, ketika Anda menjalankan *query*, BigQuery akan membaca data langsung dari file di GCS sesuai kebutuhan, tanpa data tersebut perlu berada di dalam penyimpanan BigQuery

▼ Analogi dengan Software Development dan DBMS

Dalam pengembangan perangkat lunak tradisional dengan DBMS (seperti PostgreSQL, MySQL), data disimpan langsung di dalam sistem database. Ketika Anda melakukan *query*, DBMS mengakses data dari penyimpanan

internalnya.

Mengapa di Data Engineering dengan BigQuery dan GCS Berbeda?

Ada beberapa alasan utama mengapa arsitektur dengan GCS sebagai penyimpanan "asli" dan BigQuery untuk *query* menjadi umum dalam *Data Engineering* skala besar:

1. Skalabilitas dan Fleksibilitas Penyimpanan:

- **GCS sangat scalable dan murah** untuk menyimpan data dalam jumlah besar dalam berbagai format. Ini bertindak sebagai *data lake* yang dapat menampung data mentah, data yang sudah diproses, dan berbagai jenis file.
- **DBMS tradisional memiliki batasan skalabilitas** dan seringkali lebih mahal untuk menyimpan data dalam skala petabyte. Mereka juga lebih ketat dalam hal format data.

2. Separasi Penyimpanan dan Komputasi:

- **GCS fokus pada penyimpanan yang tahan lama dan murah.**
- **BigQuery fokus pada *engine* komputasi yang kuat dan *serverless* untuk analisis.** Dengan memisahkan penyimpanan dan komputasi, Anda dapat menskalakan keduanya secara independen sesuai kebutuhan. Misalnya, Anda dapat memiliki banyak data di GCS tetapi hanya menggunakan sumber daya komputasi BigQuery saat Anda perlu melakukan analisis. Ini lebih efisien dari segi biaya dan sumber daya.

3. Fleksibilitas Format Data:

- **GCS dapat menyimpan berbagai format data** tanpa perlu transformasi awal.
- **BigQuery dapat langsung *query* berbagai format data** yang umum digunakan dalam *data lake* (CSV, JSON, Parquet, Avro) melalui *external table*. Ini menghilangkan kebutuhan untuk memuat dan mengubah semua data ke dalam format tabel BigQuery sebelum dianalisis.

4. Efisiensi Biaya:

- **Penyimpanan di GCS umumnya lebih murah** daripada penyimpanan aktif di BigQuery.
- **BigQuery mengenakan biaya berdasarkan data yang diproses oleh *query*.** Dengan menggunakan *external table*, Anda hanya membayar untuk data yang benar-benar dibaca selama *query*, bukan untuk seluruh data yang disimpan di GCS.

5. Manajemen Data yang Lebih Mudah untuk Beberapa Kasus:

- Dalam beberapa skenario, data mungkin dihasilkan atau dikumpulkan oleh sistem lain dan langsung disimpan di GCS. Membuat *external table* memungkinkan Anda untuk menganalisis data ini di BigQuery tanpa perlu melalui proses ETL (Extract, Transform, Load) yang kompleks untuk memindahkannya ke penyimpanan BigQuery.

▼ Kapan Menggunakan External Table?

- **Menganalisis data yang sudah ada di GCS** tanpa perlu memindahkannya ke BigQuery.
- **Bekerja dengan format data yang berbeda** yang mungkin tidak secara native dioptimalkan untuk penyimpanan BigQuery (meskipun BigQuery mendukung banyak format secara efisien).
- **Mengurangi biaya penyimpanan BigQuery** untuk data yang jarang di-*query*.
- **Memungkinkan berbagai tim atau proses untuk menghasilkan data ke GCS**, dan tim analis dapat langsung melakukan *query* menggunakan BigQuery.

Keterbatasan External Table:

- **Performa *query* mungkin sedikit lebih lambat** dibandingkan dengan *query* pada tabel native BigQuery karena BigQuery perlu membaca data dari GCS setiap kali *query* dijalankan.
- **Beberapa fitur BigQuery mungkin tidak sepenuhnya didukung** untuk *external table*.
- **Keandalan *query* bergantung pada ketersediaan dan integritas data di GCS.**

Kesimpulan:

Konsep *external table* di BigQuery adalah jembatan yang memungkinkan Anda memanfaatkan penyimpanan murah dan fleksibel dari GCS sambil tetap menggunakan kekuatan *query* SQL dan skalabilitas komputasi BigQuery. Ini adalah pola

yang umum dalam arsitektur *data lake* modern, di mana data mentah dan terolah disimpan di penyimpanan objek seperti GCS, dan alat analisis seperti BigQuery digunakan untuk mendapatkan *insight* dari data tersebut sesuai kebutuhan. Ini memberikan fleksibilitas, skalabilitas, dan efisiensi biaya yang lebih baik dibandingkan menyimpan seluruh data di dalam sistem database analitik.